

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Tacu Igor
FAF - 242

REPORT

Laboratory Works No. 4 and No. 3

Command Line Processing and the xv6 Shell

Course: Operating Systems

Track A: the real OS from the outside Linux

Checked by:
Reițman Patricia, assistent universitar

UTM Chișinău - 2026

1 Purpose of the Laboratory Work

The purpose of this laboratory work is to develop practical fluency with the Bash command line. The work examines command discovery, shell expansion, quoting, redirection, exit statuses, command chaining, background jobs, text-filter pipelines, and executable shell scripts. The experiments were performed on a remote Ubuntu Linux server as the root user.

2 Problem Conditions

Requirements and conditions:

1. Part 1 - use completion and history, distinguish built-ins from external commands, inspect PATH, and use local help.
2. Part 2 - predict and observe pathname expansion, quoting, brace expansion, arithmetic expansion, and command substitution.
3. Part 3 - redirect standard output and standard error, compare redirection order, use command-list operators, and control a background job.
4. Part 4 - answer Q1-Q8 with one-line pipelines stored in oneliners.sh and verify the generated 500-line access log.
5. Part 5 - implement report.sh, validate its output, and test its missing-file error path.

Preparation

The artificial access log was generated in /root/os-lab3. The first three records had the required five-field format, and wc confirmed exactly 500 lines.

```
root@ubuntu-4gb-hel1-1:~# mkdir -p ~/os-lab3
cd ~/os-lab3

for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log

head -3 access.log
wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 0.1 - Generation and verification of access.log

3 Part 1 - Move Fast and Find Help

Tab completion expanded /usr/sh to /usr/share/ and /et to /etc/. Reverse history search with Ctrl+R located and re-ran the earlier pwd command.

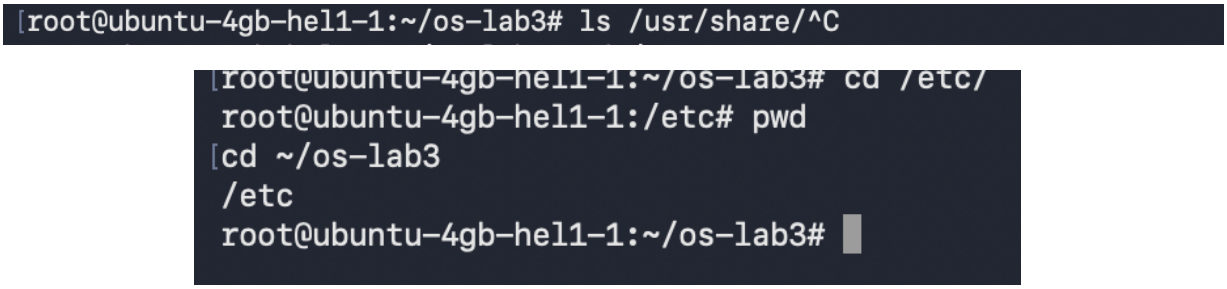


Figure 1.1 - Tab completion for /usr/share and /etc

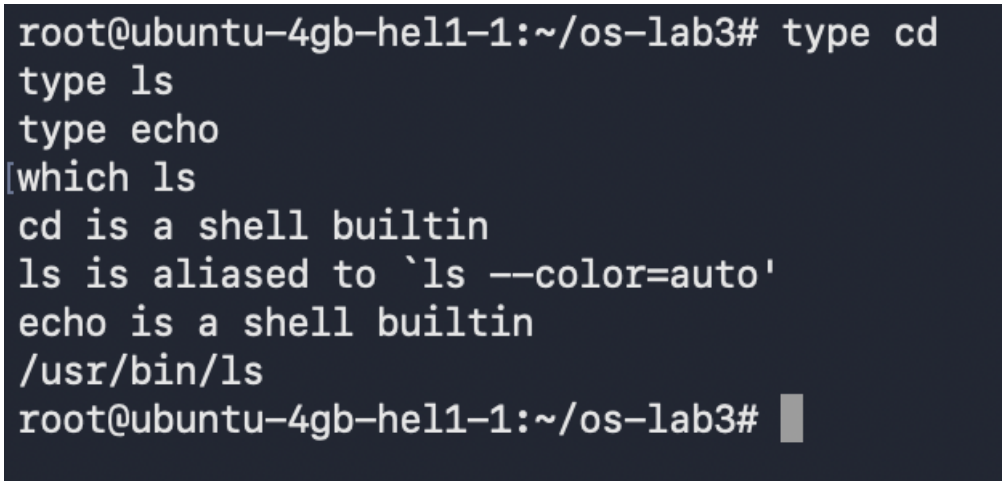


Figure 1.2 - Output of type and which

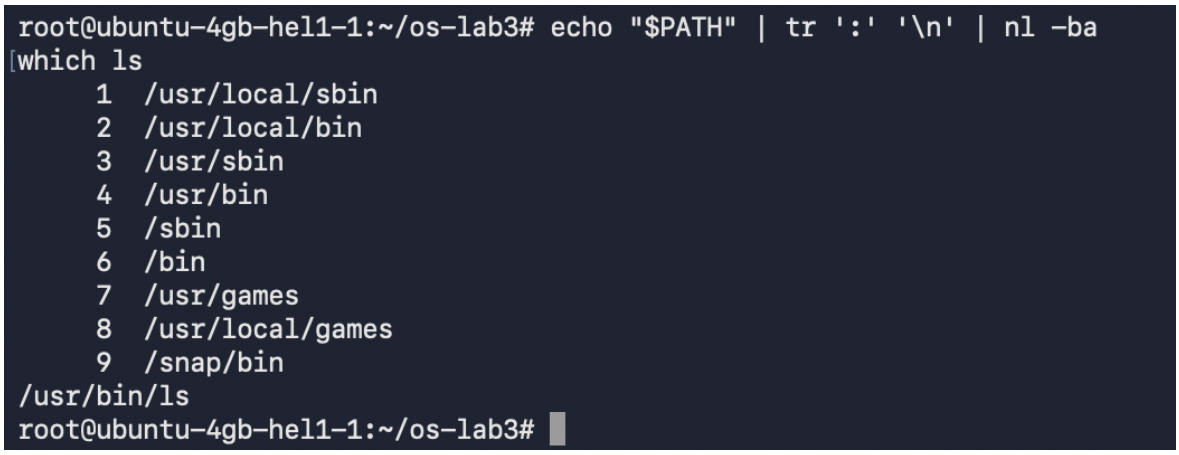


Figure 1.3 - Directories in PATH and location of ls

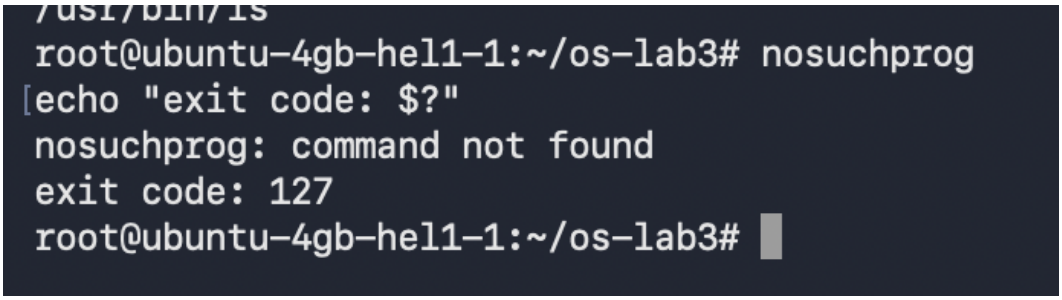


Figure 1.4 - Exit code of a missing command

Observe

1. cd and echo are Bash built-ins. ls is configured as an alias for ls --color=auto and ultimately resolves to the external program /usr/bin/ls. cd must be a built-in because changing directory must modify the working directory of the current shell; a separate child process cannot change its parent's directory.
2. PATH contains 9 directories. The ls executable is in /usr/bin, which is the fourth directory in the displayed PATH.
3. A command that is not found returns exit status 127.
4. The ls option that sorts by modification time is -t.

4 Part 2 - What the Shell Does to a Command Line

A controlled play directory was created with C files, numbered text files, a file whose name contains a space, and later three log files. Predictions were recorded before the commands were executed.

```
root@ubuntu-4gb-hel1-1:~/os-lab3# mkdir -p ~/os-lab3/play
cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
root@ubuntu-4gb-hel1-1:~/os-lab3/play#
```

Figure 2.1 - Files created in the play directory

```
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.t
root@ubuntu-4gb-hel1-1:~/os-lab3/play# echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
echo {1..5}
echo $((7 * 6))
touch log{1..3}.txt
ls log*
a.c b.c
file1.txt file2.txt
a.c alfa beta.txt b.c
*.xyz
1 2 3 4 5
42
log1.txt  log2.txt  log3.txt
root@ubuntu-4gb-hel1-1:~/os-lab3/play#
```

Figure 2.2 - Globbing brace expansion and arithmetic expansion

```
root@ubuntu-4gb-hel1-1:~/os-lab3/play# ls alfa beta.txt
ls "alfa beta.txt"
ls alfa\ beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
'alfa beta.txt'
'alfa beta.txt'
root@ubuntu-4gb-hel1-1:~/os-lab3/play#
```

Figure 2.3 - Error on an unquoted filename and two correct forms

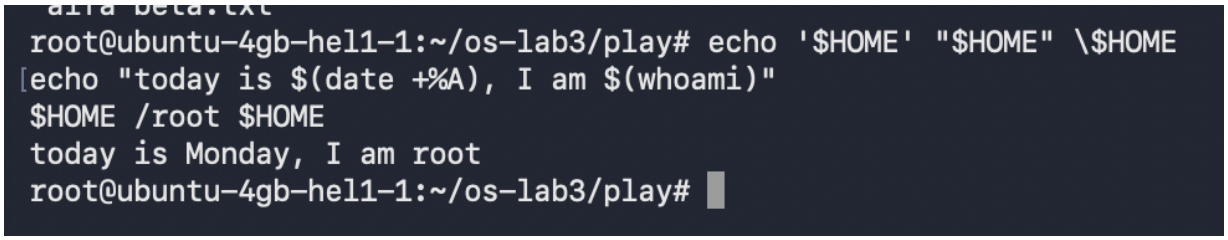


Figure 2.4 - Single quotes double quotes and backslash escaping

Prediction and Result Table

Command	Prediction	Real output	Explanation
echo *.c	a.c b.c	a.c b.c	Matched. Bash expanded the pattern before echo ran.
echo file?.txt	file1.txt file2.txt	file1.txt file2.txt	Matched. ? matched exactly one character.
echo [a-b]*	a.c alfa beta.txt b.c	a.c alfa beta.txt b.c	Matched. The bracket expression selected names beginning with a or b.
echo *.xyz	*.xyz	*.xyz	Matched. With no match, Bash left the pattern unchanged.
ls alfa beta.txt	Two missing-file errors	Two missing-file errors	Matched. The unquoted space split the intended name into two arguments.
ls "alfa beta.txt"	alfa beta.txt	'alfa beta.txt'	The file matched; ls added display quotes to make the embedded space visible.
echo '$HOME' "$HOME" \$HOME	$HOME /root $HOME	$HOME /root $HOME	Matched. Only the double-quoted variable expanded.
echo {1..5}; echo $((7 * 6))	1 2 3 4 5; then 42	1 2 3 4 5; then 42	Matched. Bash performed brace and arithmetic expansion.
echo today is ...	today is Monday, I am root	today is Monday, I am root	Matched. The subshells supplied date and user values.
touch log{1..3}.txt && ls log*	log1.txt log2.txt log3.txt	log1.txt log2.txt log3.txt	Matched. Brace expansion created three names, then globbing listed them.

Observe

1. The predictions and observed outputs are compared in the table above. The only visible difference was ls adding display quotes around the filename containing a space.
2. echo does not inspect the filesystem. Bash performed pathname expansion and replaced *.c with the matching filenames before invoking echo.
3. ls alfa beta.txt failed because the unquoted space produced two arguments: alfa and beta.txt. The filename can be protected with double quotes (ls "alfa beta.txt") or by escaping the space (ls alfa\ beta.txt).
4. Single quotes preserve every character literally. Double quotes preserve whitespace while still allowing parameter and command substitution. A backslash protects the immediately following special character.

GNU ls displayed 'alfa beta.txt' in quotes so that the embedded space was visually unambiguous. The quotation marks are presentation syntax and are not part of the stored filename.

5 Part 3 - Redirection Chaining and Jobs

```
root@ubuntu-4gb-hel1-1:~/os-lab3/play# cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt
cat err.txt
215 out.txt
ls: cannot access '/nope': No such file or directory
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 3.1 - Standard output and standard error in separate files

```
root@ubuntu-4gb-hel1-1:~/os-lab3# ls /etc /nope > all.txt 2>&1
echo "First order - beginning of all.txt:"
head -3 all.txt

ls /etc /nope 2>&1 > all.txt
echo "Second order - beginning of all.txt:"
head -3 all.txt
First order - beginning of all.txt:
ls: cannot access '/nope': No such file or directory
/etc:
GeoIP.conf
ls: cannot access '/nope': No such file or directory
Second order - beginning of all.txt:
/etc:
GeoIP.conf
PackageKit
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 3.2 - Comparing the order of redirections

```
root@ubuntu-4gb-hel1-1:~/os-lab3# ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
false ; echo "semicolon runs regardless"
false && echo "this does not run"
false || echo "OR runs after failure"
true && echo "AND runs after success"
failed with code 2
I am in /root/os-lab3/tmp
semicolon runs regardless
OR runs after failure
AND runs after success
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 3.3 - Semicolon AND and OR command-list operators

```
[1]+  Running                 sleep 100 &
root@ubuntu-4gb-hel1-1:~/os-lab3# sleep 100 &
jobpid=$!
jobs
kill "$jobpid"
wait "$jobpid"
echo "job ended with exit code $?"
jobs
[2] 3225139
[1]   Terminated              sleep 100
[2]+  Running                 sleep 100 &
[2]+  Terminated              sleep 100
job ended with exit code 143
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 3.4 - Running and terminating a background job

Observe

1. The directory listing, which was standard output, went to out.txt. The missing-path diagnostic, which was standard error, went to err.txt.
2. In > all.txt 2>&1, stdout is redirected first and stderr is then duplicated to the redirected stdout, so both streams enter all.txt. In 2>&1 > all.txt, stderr is first duplicated to the original terminal stdout; stdout is redirected afterward, so the error remains visible in the terminal while the listing enters the file.
3. wc -l < access.log reads anonymous standard input and therefore prints only 500. wc -l access.log receives a filename argument and reports both the count and the filename.
4. ; runs the next command regardless of status. && runs the next command only after success. || runs the next command only after failure.

The overwrite operator > replaced notes.md with the first line, while >> appended the second line. The background command returned control immediately; jobs displayed its state, and termination produced exit code 143, which is 128 plus signal 15 (SIGTERM).

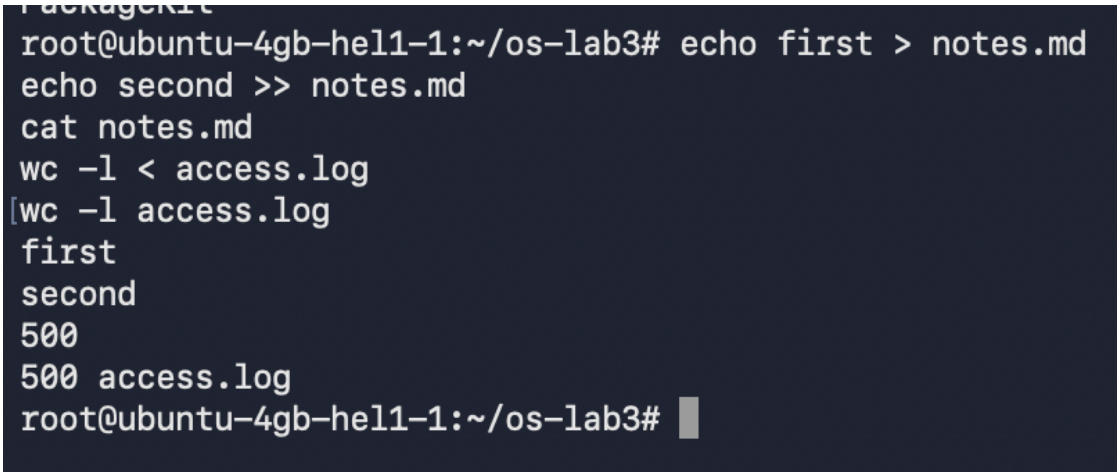


Figure 3.5 - Append redirection and the two wc input forms

6 Part 4 - Answer Questions with One Liners

Each question was answered by a single shell line using small text filters connected by pipelines. The generated log contained 500 requests.

Q1. Failed requests: 100.

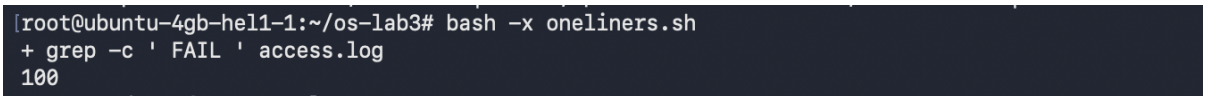


Figure 4.1 - Number of failed requests

Q2. Four pages were requested: /page0, /page1, /page2, and /page3.



Figure 4.2 - Number and names of requested pages

Q3. Each page received 125 requests; the counts sum to 500.



Figure 4.3 - Request count per page

Q4. user3 and user5 tied for the largest number of failures, with 15 each.

```
+ grep ' FAIL ' access.log
+ uniq -c
+ cut '-d ' -f3
+ sort
+ sort -rn
+ awk 'NR==1 {max=$1} $1==max'
     15 user5
     15 user3
```

Figure 4.4 - Users with the most failed requests

Q5. user3 made 72 requests, of which 15 failed.

```
++ grep -c ' user3 ' access.log
++ grep ' user3 ' access.log
++ grep -c ' FAIL '
+ printf 'user3: %s requests, %s failed\n' 72 15
user3: 72 requests, 15 failed
```

Figure 4.5 - Requests and failures for user3

Q6. The last three failed requests were 10:10 user0, 10:15 user5, and 10:20 user3.

```
+ grep ' FAIL ' access.log
+ tail -3
+ cut '-d ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 - Last three failed requests

Q7. /usr/sbin/nologin was used by 28 accounts, /bin/false by 3, /bin/sync by 1, and /bin/bash by 1.

```
+ cut -d: -f7 /etc/passwd
+ sort
+ uniq -c
+ sort -rn
     28 /usr/sbin/nologin
      3 /bin/false
      1 /bin/sync
      1 /bin/bash
```

Figure 4.7 - Login shells and account counts

Q8. ls /etc produced 214 lines and ls -l /etc produced 215. The long listing adds a total line before the entries.

```
+ printf 'ls /etc: '
ls /etc: + ls /etc
+ wc -l
214
+ printf 'ls -l /etc: '
ls -l /etc: + wc -l
+ ls -l /etc
215
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 4.8 - Comparing short and long ls output counts

```
root@ubuntu-4gb-hel1-1:~/os-lab3# bash -x oneliners.sh
+ grep -c ' FAIL ' access.log
100
+ cut '-d ' -f5 access.log
+ sort -u
+ awk '{pages[NR]=$0} END {print NR " different pages"; for (i=1;i<=NR;i++) print pages[i]}'
4 different pages
/page0
/page1
/page2
/page3
+ cut '-d ' -f5 access.log
+ sort
+ uniq -c
+ sort -rn
    125 /page3
    125 /page2
    125 /page1
    125 /page0
+ grep ' FAIL ' access.log
+ uniq -c
+ cut '-d ' -f3
+ sort
+ sort -rn
+ awk 'NR==1 {max=$1} $1==max'
     15 user5
     15 user3
++ grep -c ' user3 ' access.log
++ grep ' user3 ' access.log
++ grep -c ' FAIL '
+ printf 'user3: %s requests, %s failed\n' 72 15
user3: 72 requests, 15 failed
+ grep ' FAIL ' access.log
+ tail -3
+ cut '-d ' -f2,3
10:10 user0
10:15 user5
10:20 user3
+ cut -d: -f7 /etc/passwd
+ sort
+ uniq -c
+ sort -rn
     28 /usr/sbin/nologin
      3 /bin/false
      1 /bin/sync
      1 /bin/bash
+ printf 'ls /etc: '
ls /etc: + ls /etc
+ wc -l
214
+ printf 'ls -l /etc: '
ls -l /etc: + wc -l
+ ls -l /etc
215
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 4.9 - Full traced run of oneliners.sh

Contents of oneliners.sh

```
#!/bin/bash

# Q1. How many requests failed?
grep -c ' FAIL ' access.log

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u | awk '{pages[NR]=$0} END {print NR " different pages"; for
(i=1;i<=NR;i++) print pages[i]}'

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn

# Q4. Which user or users have the most failed requests, and how many?
grep ' FAIL ' access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1 {max=$1}
$1==max'

# Q5. How many requests did user3 make, and how many failed?
printf 'user3: %s requests, %s failed\n' "$(grep -c ' user3 ' access.log)" "$(grep ' user3 '
access.log | grep -c ' FAIL ')"

# Q6. Print the last 3 failed requests, showing only time and user.
grep ' FAIL ' access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each?
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn

# Q8. Compare the line counts.
printf 'ls /etc: '; ls /etc | wc -l; printf 'ls -l /etc: '; ls -l /etc | wc -l
```

7 Part 5 - The First Shell Script

report.sh validates its input, summarizes the complete log, and uses a loop to print the failed-request count for user0 through user6.

```
exit code: 128
[root@ubuntu-4gb-hel1-1:~/os-lab3# nl -ba report.sh
     1  #!/bin/bash
     2  # usage: ./report.sh access.log
     3
     4  log=$1
     5
     6  if [ ! -f "$log" ]; then
     7      echo "no such file: $log" >&2
     8      exit 1
     9  fi
    10
    11  total=$(wc -l < "$log")
    12  fails=$(grep -c FAIL "$log")
    13
    14  echo "$total requests, $fails failed"
    15
    16  for i in {0..6}; do
    17      count=$(grep -c " user$i FAIL " "$log")
    18      echo "user$i: $count failures"
    19  done
root@ubuntu-4gb-hel1-1:~/os-lab3#
```

Figure 5.1 - Contents of report.sh

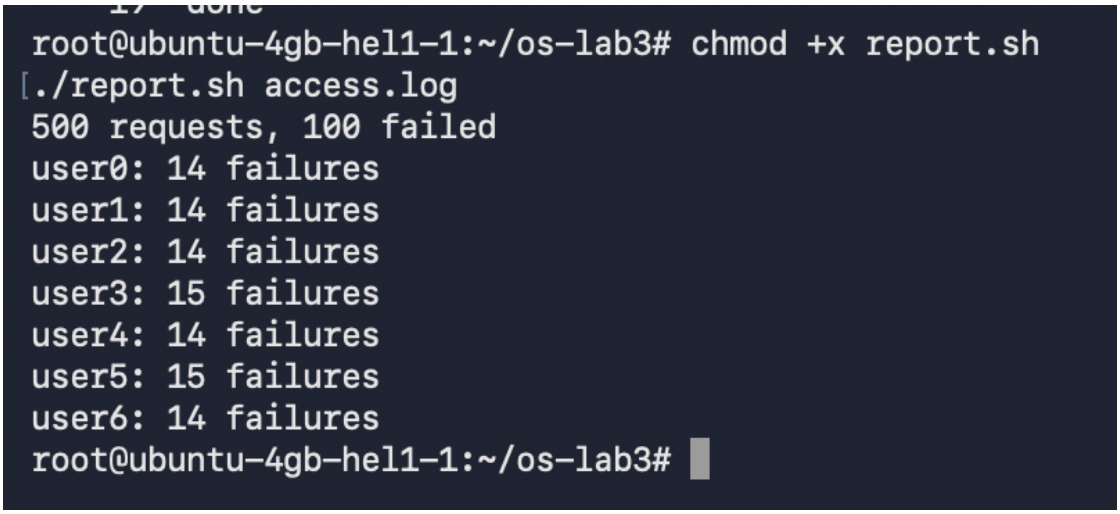


Figure 5.2 - Running report.sh on access.log

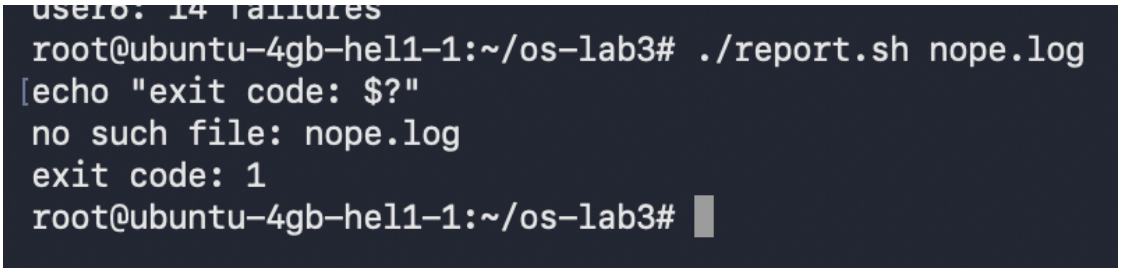


Figure 5.3 - Error handling for a missing input file

Observe

1. #!/bin/bash is the shebang. It tells the operating system to use /bin/bash as the interpreter when the file is executed directly. Before chmod +x, direct execution failed with Permission denied and exit status 126.
2. ./report.sh is required because the current directory is not normally included in PATH. The ./ prefix supplies an explicit path to the executable.
3. The script writes its diagnostic to standard error with >&2 so normal output and errors can be redirected independently. exit 1 returns a nonzero failure status to the calling shell or automation tool.

Source of report.sh

```
#!/bin/bash
# usage: ./report.sh access.log

log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

for i in {0..6}; do
    count=$(grep -c " user$i FAIL " "$log")
    echo "user$i: $count failures"
done
```

8 Conclusion and Critical Analysis

Summary of work:

- Used Bash completion, command history, built-ins, PATH inspection, exit codes, and local help.
- Observed globbing, quoting, brace expansion, arithmetic expansion, and command substitution.
- Separated stdout and stderr, compared left-to-right redirection semantics, applied ;, &&, and ||, and controlled a background process.
- Built verified pipelines for log analysis and stored them in an executable one-liner collection.
- Implemented report.sh with input validation, summary calculations, a user loop, and explicit failure handling.

Critical analysis:

- The shell transforms a command before the target program receives it. The globbing exercises showed that echo never searched the directory; Bash supplied the expanded argument list.
- Quoting is part of correctness, not presentation. A single unquoted space changed one valid pathname into two invalid arguments, while quotes or a backslash preserved the intended filename.
- Redirections are evaluated from left to right. The same tokens in a different order changed whether stderr followed stdout into a file or remained attached to the terminal.
- Pipelines make complex questions manageable because each filter performs one narrow transformation. Verification invariants were important: the page totals summed to 500, and per-user failure totals summed to 100.
- Exit statuses make scripts composable. Status 127 identified command lookup failure, 126 identified a non-executable file, 143 identified SIGTERM, and status 1 signaled the script's deliberate missing-file failure.

Overall, the laboratory demonstrated how Bash combines parsing, expansion, process control, file descriptors, and small Unix filters into a practical interface to the operating system.

9 Laboratory Work No. 4 The xv6 Shell from the Outside

9.1 Purpose and Requirements

The purpose of Laboratory Work No. 4 is to compare the compact xv6 shell with Bash, verify the sleep user program against the current xv6 system-call interface, observe which shell performs argument expansion, and identify the five command structures implemented by user/sh.c.

The work was performed on the remote Ubuntu server as root. The existing ~/xv6-riscv tree was found and used successfully.

- Update and test user/sleep.c with the pause system call declared by the current tree.
- Run the same redirection, pipe, list, background, glob, variable, quote, path, and interactive tests in Bash and xv6.
- Create xv6 and Linux versions of args, then compare the exact argv arrays received by each program.
- Read user/sh.c to identify command types, execution cases, cd handling, and missing expansion logic.

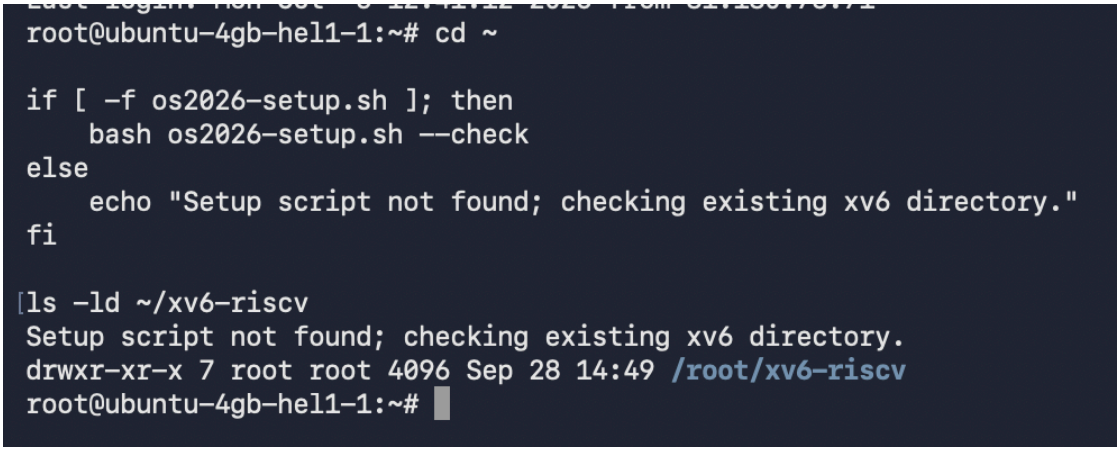


Figure 9.1 - Verification of the existing xv6-riscv working tree

10 Lab 4 Part 1 Fix sleep for the Current xv6

The header search found int pause(int) in user/user.h. Therefore user/sleep.c was updated to call pause(atoi(argv[1])). The existing $U/_sleep entry in UPROGS was retained, and the source built successfully.

```
drwxr-xr-x 7 root root 4096 Sep 28 14:49 /.
root@ubuntu-4gb-hel1-1:~# cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
25:int pause(int);
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 10.1 - The pause system-call declaration in user.h

```
$U/_sleep\ is already present
root@ubuntu-4gb-hel1-1:~/xv6-riscv# nl -ba user/sleep.c
grep -n -B2 -A3 '\$U/_sleep' Makefile
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char *argv[])
     6  {
     7      if(argc != 2){
     8          fprintf(2, "usage: sleep <ticks>\n");
     9          exit(1);
    10      }
    11
    12      pause(atoi(argv[1]));
    13      exit(0);
    14  }
133-    $U/_cat\
134-    $U/_echo\
135:    $U/_sleep\
136-    $U/_forktest\
137-    $U/_grep\
138-    $U/_init\
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 10.2 - Updated sleep.c and the UPROGS entry

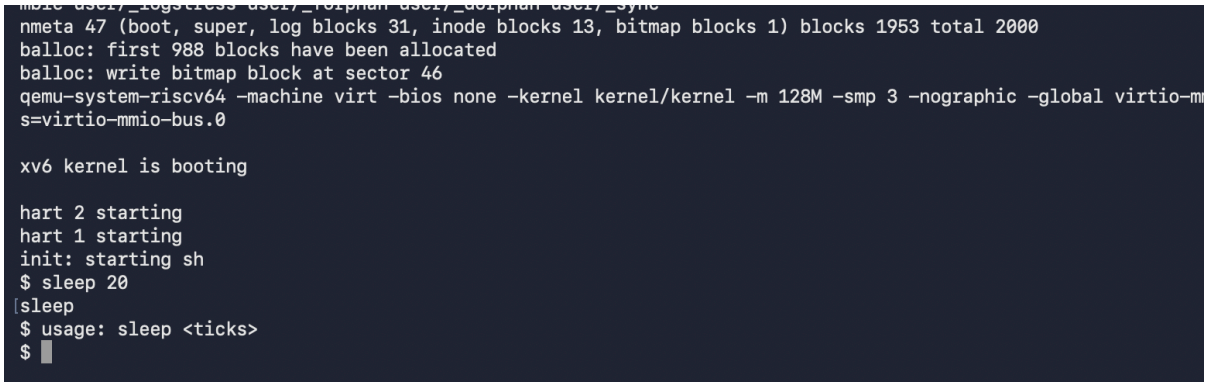


Figure 10.3 - sleep running in xv6 with and without an argument

Observe

1. The current xv6 tree exposes pause(int), so a Lab 2 implementation that calls the former sleep system call must be adjusted.
2. sleep 20 returned to the prompt after the requested delay, confirming that the program was linked into the file-system image and invoked correctly.
3. Calling sleep without an argument produced usage: sleep <ticks> and exited with a failure status, so invalid input is handled explicitly.

11 Lab 4 Part 2 Bash versus the xv6 Shell

The same command lines were executed first in Bash inside ~/shtest and then at the xv6 prompt. The table records both the visible result and the shell feature being tested.

Command	Tests	bash	xv6
echo hello > f ; cat < f	Redirection	Prints hello	Prints hello
ls | grep READ	Pipe	README	README
(echo a; echo b) | wc	Grouping and list	2 lines, 2 words, 4 bytes	Same pipeline behavior
echo one ; echo two	Sequence	one, then two	one, then two
echo bg &	Background	Job information and bg	Runs without waiting; no job-control UI
ls *	Globbing	Expands to README and f	Literal *; ls reports it cannot open *
echo $HOME	Variables	Prints /root	Prints literal $HOME
echo "a b"	Quotes	Prints a b without quotes	Prints the quote characters: "a b"
mkdir d ; cd d ; ls ..	PATH / lookup	ls is found through PATH	Relative ls fails after cd; /ls .. works
Up, Tab, Ctrl-R	History/completion	Available	Not implemented

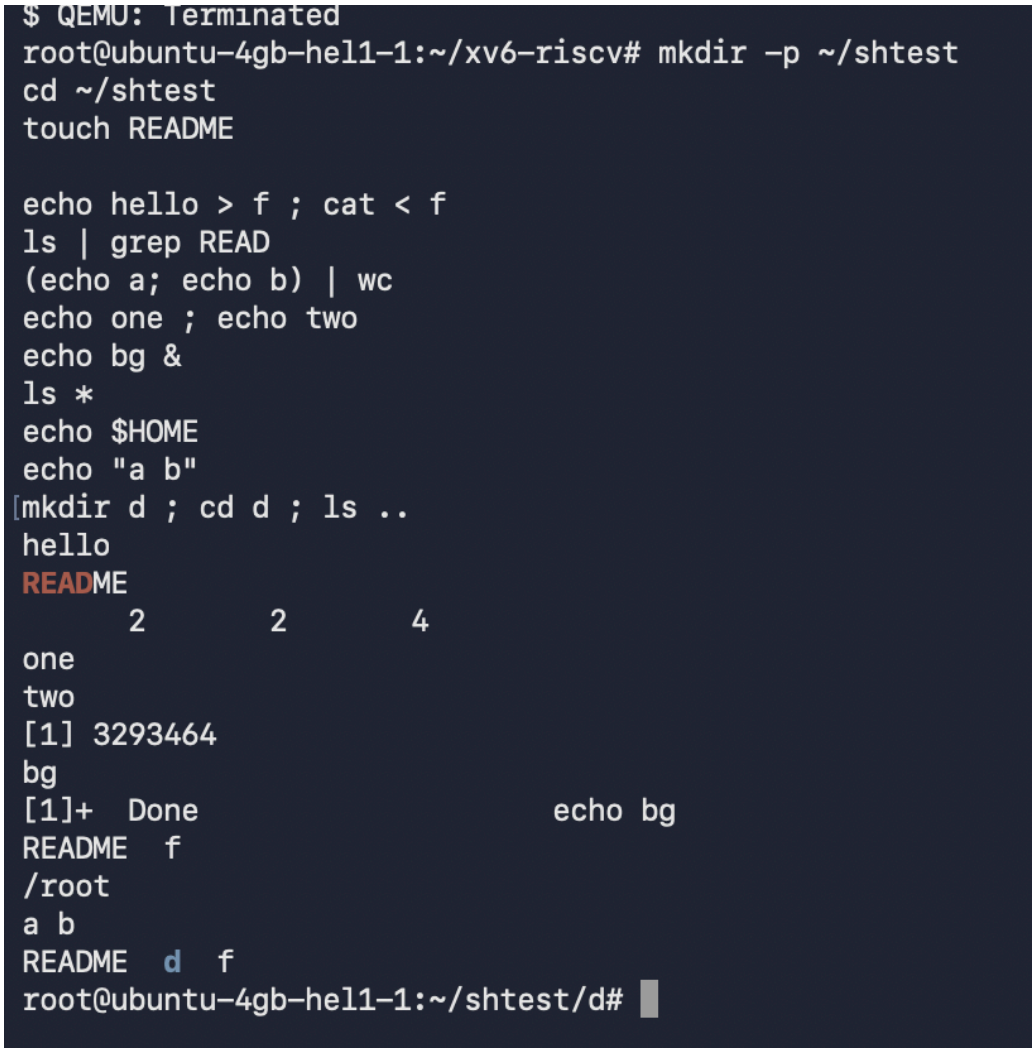


Figure 11.1 - Commands executed in Bash

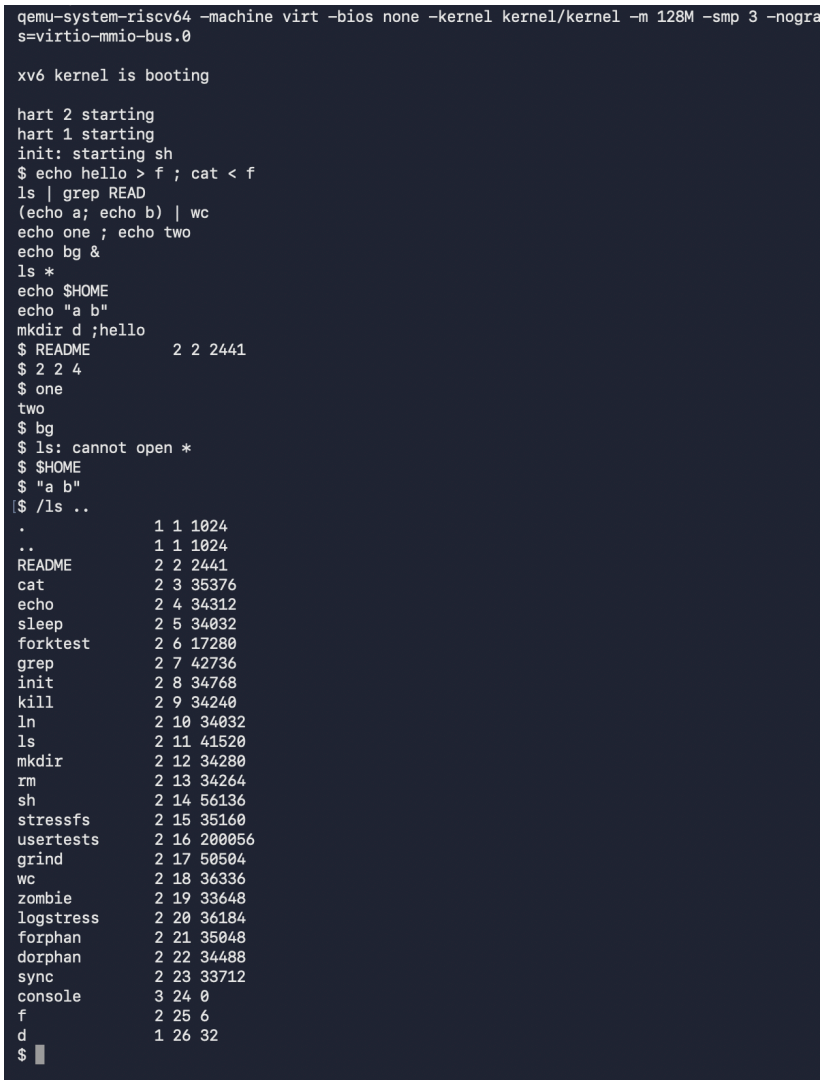


Figure 11.2 - The same shell-feature tests executed in xv6

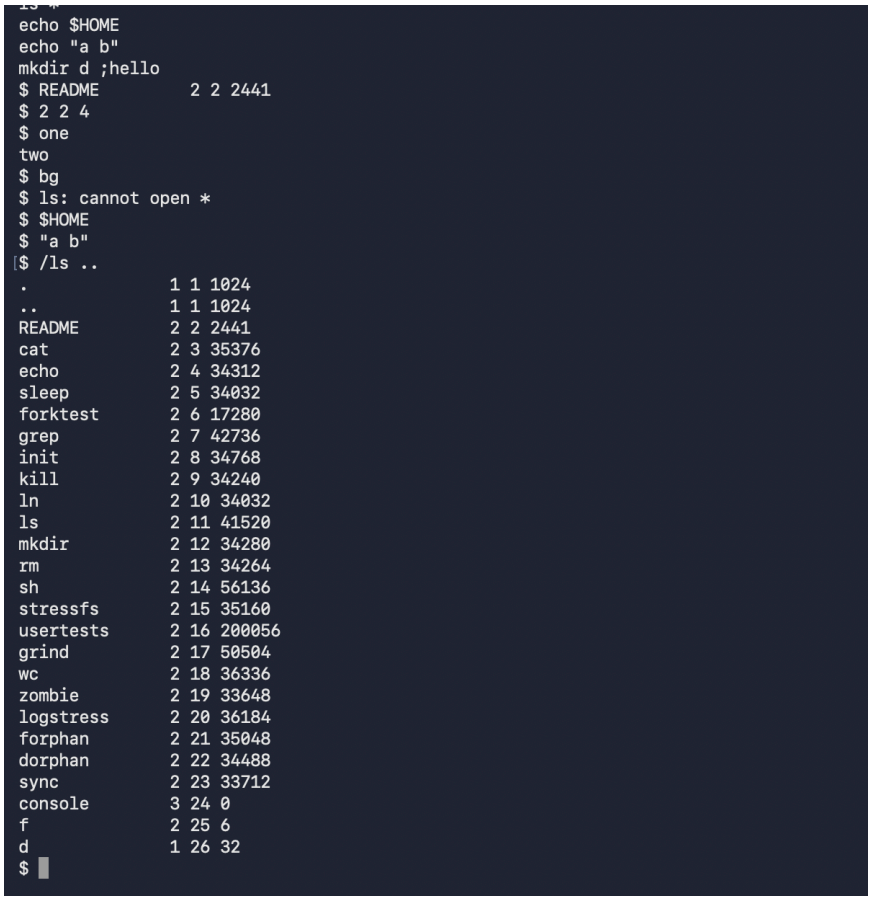


Figure 11.3 - Literal wildcard behavior and running /ls by absolute path

Observe

1. Both shells support ordinary command execution, input and output redirection, pipes, semicolon-separated lists, parenthesized command groups, and background execution. Bash additionally provides glob expansion, environment-variable expansion, quote removal, PATH lookup, command history, interactive search, completion, and full job-control feedback.
2. After cd d in xv6, a command named ls is searched for relative to the new directory because the xv6 shell does not implement PATH lookup. The executable resides in the root directory of the xv6 file system, so /ls .. runs it successfully from inside d.
3. In daily work, command history and tab completion would be missed most because they accelerate almost every interactive task. Basic quote recognition would be one of the easiest missing features to add because it can be implemented in tokenization without requiring directory scanning or an environment-variable subsystem.

12 Lab 4 Part 3 Who Expands the Arguments

Two small programs were used to display argc and every argv element: user/args.c for xv6 and args_linux.c for Linux. The programs contain no expansion logic; they only print the values already prepared by their invoking shell.

Source of user/args.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);
    exit(0);
}
```

The program was added to UPROGS as $U/_args\ so that it was included in the xv6 file-system image.

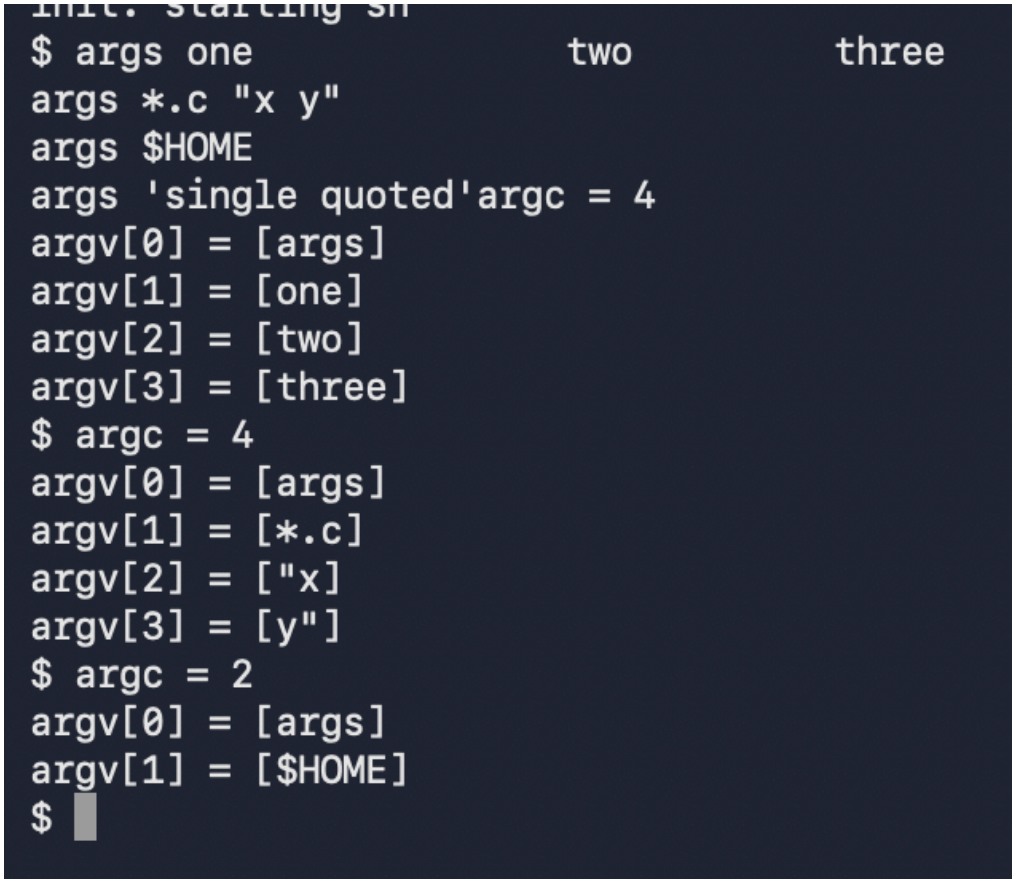


Figure 12.1 - Argument arrays received by args in xv6

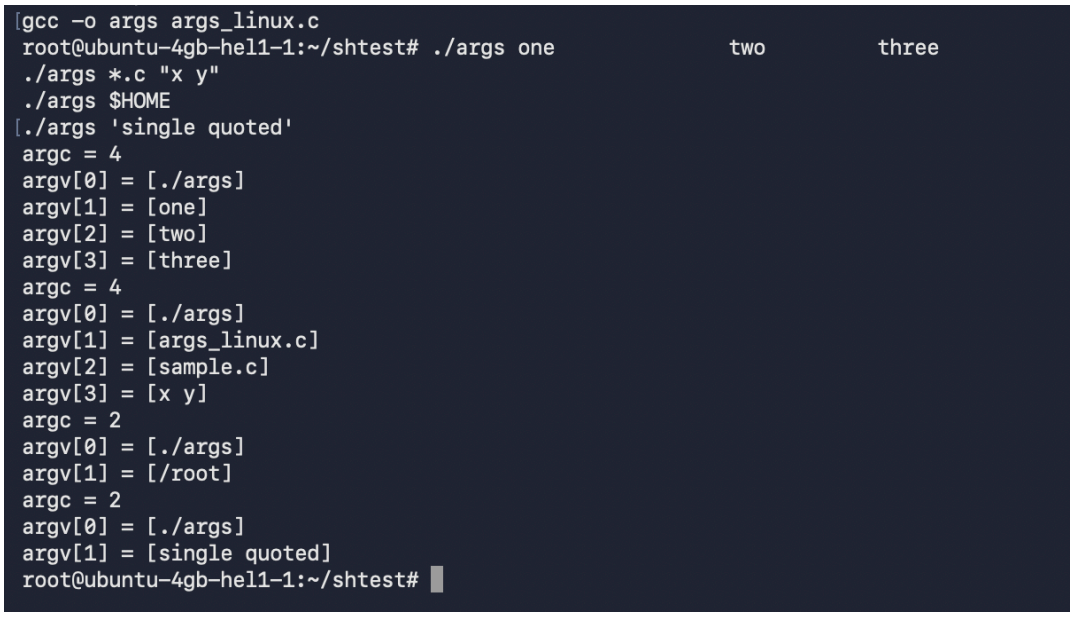


Figure 12.2 - Argument arrays received by args_linux in Bash

Observe

1. For args *.c "x y", Bash supplied four arguments in total: ./args, args_linux.c, sample.c, and the single argument x y. xv6 also supplied four arguments, but they were args, the literal *.c, "x, and y".
2. Bash, not the program or the kernel, turns *.c into matching file names and removes syntactic quotes before exec. The minimal xv6 shell performs neither operation, so the literal characters reach args.
3. The additional spaces in args one two three were treated only as separators. Both shells delivered the same three words without preserving the repeated spacing.
4. Bash expanded $HOME to /root. xv6 passed the two characters $ and the name HOME as one literal argument.

13 Lab 4 Part 4 Reading the xv6 Shell

user/sh.c contains 499 lines in this tree. Its definitions and execution switch show that the shell represents a command as one of five structures.

```
499 user/sh.c
root@ubuntu-4gb-hel1-1:~/xv6-riscv# grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 13.1 - Command kinds defined in user/sh.c

```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 13.2 - Switch cases that execute and construct command kinds

```
493:  case BACK:
root@ubuntu-4gb-hel1-1:~/xv6-riscv# grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 13.3 - Location of the diagnostic in the cd handling code

```
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
root@ubuntu-4gb-hel1-1:~/xv6-riscv# grep -nF -e "'*'" -e "'$'" user/sh.c || echo "No explicit * or $ expansion code found"
No explicit * or $ expansion code found
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 13.4 - Search confirming no explicit glob or variable expansion

Observe

1. The five command kinds are EXEC for an ordinary command, REDIR for < or >, PIPE for |, LIST for ;, and BACK for &. Parentheses group a parsed command but do not introduce a sixth execution kind.
2. cd is recognized in the shell's main loop and chdir is called by the shell process itself before the usual fork-and-run path. If cd were a separate child program, only the child's working directory would change and the interactive shell would remain in its old directory.
3. The source contains no code that expands * into directory entries or $NAME into an environment value. This directly explains why xv6 passed *.c and $HOME literally in Parts 2 and 3.

14 Lab 4 Conclusion and Critical Analysis

Summary of work:

- Updated and verified the sleep program for the pause system call used by the current xv6 tree.
- Compared Bash and xv6 behavior for redirection, pipes, grouping, lists, background execution, globbing, variables, quotes, path lookup, history, and completion.
- Built argument-printing programs for xv6 and Linux and used them to identify the shell as the component that performs expansion and quote removal.
- Read user/sh.c and identified its five command structures, its special handling of cd, and its lack of glob and variable expansion.

Critical analysis:

- The experiment separates parsing from execution. args produced different arrays despite equivalent source code because each shell transformed the input differently before exec.
- xv6 demonstrates that redirection, pipelines, lists, and asynchronous execution require relatively little shell code when advanced interactive and expansion features are omitted.
- The failed relative ls after changing directories illustrates that command lookup is a shell policy rather than a kernel service. An absolute executable path remains valid because it does not depend on PATH search.
- Handling cd inside the parent shell is essential state management, not merely a convenience. The same principle applies to other shell state such as variables and working-directory history.

Overall, the work showed that the kernel executes the final argv array, while the shell determines much of the user-visible command language. Bash adds expansion, quoting rules, search paths, history, completion, and job control; xv6 exposes the smaller set of mechanisms needed to parse commands, connect processes, redirect file descriptors, and manage essential shell state.